

Convolution lowering과 Matrix tiling을 적용할 때 변수들의 순서는 강의자료를 참고하였다.

1. Convolution lowering

Convolution lowering은 convolution 연산을 행렬 곱 연산으로 바꾸어 주는 방법으로, input image와 kernel을 matrix로 만들어야 한다.

kernel은 C_o 개의 3차원 tensor로 구성된다. 즉, $H_k \times W_k \times C_I \times C_o$ 로 표현할 수 있었지만, 이를 행렬, 즉 2차원 matrix $(C_I \times H_k \times W_k) \times C_o$ 로 변환해주어야 한다. 즉, `weight[co].reshape(-1)`를 통해 나머지 차원을 펴줌으로써 kernel matrix를 저장한다.

Input image는 B(Batch size)개의 3차원 tensor로, $B \times H_I \times W_I \times C_I$ 의 형태를 가진다. 이를 lowering하면 행렬, 즉 2차원 matrix인 $(H_o \times W_o) \times (C_I \times H_k \times W_k)$ 로 재구성하게 된다. 이때 Input image의 크기를 계산하는 데 사용되는 변수가 바뀌고 개수가 늘어났음을 확인할 수 있는데, 이는 Input image를 단순히 펼치는 것이 아니라 커널 위치에 있는 Input image 값들을 한 행으로 flatten하며 저장하고, 이 과정에서 중복 저장이 일어나기 때문이다.

우선 위 수식에 나와 있는 변수 순서대로 for 루프를 만들고, 각 출력 위치마다 Input image의 해당 receptive field를 하나의 row로 펼친다. 이후 다음과 같은 코드로 lowering을 진행한다.

```
input_lowering[row, col] = imagePadded[batch, ci, h * H_s + hk, w * W_s + wk]
```

imagePadded의 인덱스를 해석해보면, **batch**는 Batch 인덱스이고 **ci**, **$h * H_s$** , **$w * W_s$** 는 각각 input channel 인덱스, 현재 출력 위치에 대응하는 입력 시작 좌표이다. 또한 **hk**, **wk**는 커널 내부 높이와 너비 offset이다. 즉, 출력 좌표 ($h=row, w=col$)에 대응하는 입력 패치(커널의 위치에 대응되는 Input image 값) 안에서 ci번째 채널의 (hk, wk) 값을 가져와 input_lowering에 저장한다고 이해할 수 있다.

이때 입력 패치에 대해 생각해보면, 커널이 입력 위를 이동할 때 같은 Input image 값이 출력의 여러 위치의 receptive field에 포함될 수 있으므로, lowering 과정에서는 Input image 값이 중복 저장되면서 행렬 크기가 커진다. 이렇게 해서 Input image matrix를 만들고, 이후 Matrix Tiling에 바로 사용할 수 있다.

2. Matrix tiling

Matrix tiling에서는 전체 행렬을 $T \times T$ 크기의 tile 단위로 부분 행렬 곱을 하고, 그 결과를 partial_sum에 누적한다. 이에 따라 기본적으로 i, j, k를 묶어서 input tile과 output tile의 시작 인덱스로 사용하고, np.matmul을 사용하지 않는 경우 ii, jj, kk를 묶어서 tile 내부 위치를 나타내는 인덱스로 사용한다. 이때 i, j, k는 행렬의 너비와 높이가 T로 나뉘 떨어지지 않을 수 있으므로 i_end, j_end, k_end를 통해 마지막 타일의 경계를 지정해주어야 한다.

i, j, k를 이용하여 input_tile, kernel_tile을 지정하는데, 주어진 과제 조건에 따라 T를 4와 64로 바꾼 두 버전의 구현을 블록으로 나누어 작성하였다. 또한 T가 tile_m, tile_n, tile_k로 따로 정의될

때를 가정하여 따로 저장하였다. 이 경우 input_tile의 너비와 kernel_tile의 높이가 tile_k로 같아야 함을 `input_tile[:tile_m, :tile_k] = 0`, `kernel_tile[:tile_k, :tile_n] = 0`을 통해 확인할 수 있다.

tile 내부의 행렬 곱은 np.matmul을 이용하지 않는 경우와 이용하는 경우에서 각각 다르게 구현되었다. np.matmul을 이용하지 않는 경우 tile 내부 원소를 ii, jj, kk 반복문으로 직접 계산하고, 각 원소의 곱을 acc 변수에 누적한 뒤 partial_sum에 더한다. 반면 np.matmul을 이용하는 경우 tile 단위 부분 행렬 곱을 라이브러리 함수로 계산하므로 acc 변수 선언 없이 바로 partial_sum에 누적한다.

3. 결과 비교 및 해석

위 2가지 구현을 기본 구현과 시간을 측정하여 비교해보았다. 제공된 파일에는 np.matmul을 사용하는 블록에만 모든 conv 연산을 측정하고, np.matmul을 사용하지 않는 블록에는 conv1과 layer1.0.conv1만 시간을 측정하도록 되어 있었다. 이를 수정하여 np.matmul을 사용하지 않는 블록에서도 모든 conv 연산을 측정하도록 하였다. 그 결과는 아래와 같다.

	original	Conv lowering + Matrix Tiling(T=4)	Conv lowering + Matrix Tiling(T=64)	Conv lowering + Matrix Tiling(T=4) + np.matmul	Conv lowering + Matrix Tiling(T=64) + np.matmul
conv1	38.79826498031616	11.974595546722412	8.9679536819458	1.5933241844177246	0.15433144569396973
Layer1.0 conv1	803.4227991104126	232.46847343444824	163.2003116607666	36.64575266838074	2.1684446334838867
Layer1.0 conv2		233.89903664588928	163.42032146453857	36.079644203186035	2.2382333278656006
Layer1.1 conv1		233.16423773765564	165.3164632320404	37.31403303146362	2.1418654918670654
Layer1.1 conv2		232.87469005584717	165.42503380775452	35.61065340042114	2.4288806915283203
Layer2.0 conv1		117.11981105804443	80.19445562362671	17.418259143829346	1.1664602756500244
Layer2.0 conv2		232.92492127418518	164.71825623512268	36.82562446594238	2.2182655334472656
Layer2.0 shortcut		13.049499273300171	9.911975383758545	1.817516565322876	0.22086668014526367
Layer2.1 conv1		235.91833567619324	163.29200100898743	35.71904754638672	1.1550297737121582
Layer2.1 conv2		231.71863389015198	163.7695915699005	36.04010009765625	1.1566638946533203
Layer3.0 conv1		118.51000165939331	82.74007797241211	17.363117456436157	0.38797593116760254
Layer3.0 conv2		231.27988004684448	160.80357670783997	36.69464588165283	0.6637918949127197
Layer3.0 shortcut		13.27424430847168	9.952612161636353	1.7532224655151367	0.10649538040161133
Layer3.1 conv1		232.30555295944214	160.3734188079834	34.541683197021484	0.6682612895965576
Layer3.1 conv2		231.67629170417786	163.55279302597046	36.52508354187012	0.6836304664611816
Layer4.0 conv1		116.91354084014893	80.72260189056396	17.249529361724854	0.33037638664245605
Layer4.0 conv2		232.85667610168457	162.0636293888092	36.336183071136475	0.7105729579925537

위 표를 간단하게 정리하면, 기본 구현보다 Convolution lowering과 Matrix tiling을 적용했을 때가 더 빠르고, np.matmul을 적용한 것이 더 빠르고, tile 사이즈를 4×4로 적용한 것이 64×64로 적용했을 때보다 빠르다고 할 수 있다.

Convolution Lowering과 Matrix Tiling을 비교해보면, 기본 구현에서는 layer1.0.conv1이 약 803.42초가 소요되었지만, T=4로 tiling한 구현에서는 layer1.0.conv1이 약 232.47초로 약 71% 감소하였다. 이는 convolution 연산을 lowering하여 GPU에서 빠르게 계산할 수 있는 행렬 곱 형태로 바꾸고, 타일 단위로 계산함으로써 효율적인 메모리 접근이 가능해졌기 때문이다.

또한 위의 232.47초는 np.matmul을 사용하지 않은 T=4일 때의 소요 시간이지만, np.matmul을 이용한 구현에서는 T=4일 때 layer1.0.conv1 계산에 약 36.65초가 사용되어 84.23% 줄어든 것을 확인할 수 있었다. 이는 numpy가 단순한 python 반복문으로 행렬 곱을 계산하는 방식보다 훨씬 빠른 방법을 이용하고 있다고 해석할 수 있다.

한편 타일 크기를 T=4와 T=64로 달리 했을 때는 T=64로 했을 때가 더 빠른 성능을 보였다. np.matmul을 사용한 layer1.0.conv1 기준 T=4일 때 36.65초가 걸린 반면, T=64일 때는 2.17초가 소요되어 약 94.08%만큼 감소했다. 이는 T=64일 때 tile size가 충분히 크면서 한번에 많은 양을 np.matmul이 빠르게 처리해주면서 시간이 단축된 것으로 추측된다.